

03. Python Fundamentals (Part-3)

Concepts: String, List, Tuple, Dictionary & Set

Author: Golam Maula Lincoln

GitHub Profile: <https://github.com/gmlincoln>

[Visit GitHub Profile](#)

1. Strings

A **string** in Python is a sequence of characters enclosed in quotes.

```
str1 = "hello world"
str2 = 'Prime'
```

Immutability: Strings are **immutable** in Python, meaning once created, their contents cannot be changed directly.

Length & Concatenation

We use `len()` to get the length of a string, and the `+` operator to concatenate (join) strings.

```
# String Length
word = 'Prime'
print(len(word)) # Output: 5

# Concatenation
str1 = "Apna"
str2 = "College"
word = str1 + " " + str2 # concatenation
print(word) # Output: Apna College
```

Looping & Indexing in Strings

Python follows **Zero-based indexing** where the first character is at index `0`. Negative indexing starts from the last character at index `-1`.

```
# Looping on Strings
s = "Python"
for ch in s:
    print(ch) # prints 'P', 'y', 't', 'h', 'o', 'n'

# Indexing
print(s[0]) # Output: 'P'
print(s[3]) # Output: 'h'
print(s[-1]) # Output: 'n' (last character)
```

Slicing in Strings

Slicing lets us access multiple elements at once using the syntax `string[start:stop:step]`.

- **start:** Index where slice begins (inclusive). Defaults to `0`.
- **stop:** Index where slice ends (exclusive). Defaults to string length.
- **step:** Number of indices to move forward each time. Defaults to `1`.

```
s = "Python"
print(s[0:2]) # Output: 'Py'
print(s[2:]) # Output: 'thon'
print(s[:3]) # Output: 'Pyt'
print(s[::-2]) # Output: 'Pto' (every second char)
print(s[::-1]) # Output: 'nohtyP' (reversed string)
```

String Formatting

String formatting creates dynamic strings by inserting values into predefined templates. The primary methods are `.format()` and f-strings (Python 3.6+).

```
# 1. Using .format()
name = "Rahul"
age = 25
text = "My name is {} and I am {} years old".format(name, age)
print(text)

# Positional & Named placeholders
print("Coordinates: {1}, {0}".format("x", "y")) # Output: Coordinates: y, x
print("Name: {n}, Age: {a}".format(n="Bob", a=30))

# 2. Using f-strings (Preferred)
name = "Rahul"
age = 25
text = f"My name is {name} and I am {age} years old"
print(text)

a = 5
b = 10
print(f"sum of {a} & {b} = {a + b}")
print(f"avg of {a} & {b} = {(a + b) / 2}")
```

2. Lists

A **list** is a collection of items in Python written using square brackets `[]`.

List Characteristics

1. **Ordered:** Items have a defined order accessible by index.
2. **Mutable:** Items can be changed after list creation.
3. **Allows Duplicates:** The same value can appear multiple times.
4. **Heterogeneous:** Can contain different data types.

```
my_list = [1, 2, 3, 4, 5]
print(type(my_list)) #

my_list2 = [10, "Hello", 3.14, True, 10] # Heterogeneous list
print(my_list2)
```

List Indexing, Modification & Slicing

```
my_list = ["apple", "banana", "cherry"]
print(my_list[0]) # Output: apple
print(my_list[-1]) # Output: cherry

# Modifying elements
my_list = [1, 2, 3, 4]
my_list[0] = 10
print(my_list) # Output: [10, 2, 3, 4]

# Slicing Examples
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(numbers[2:5]) # Output: [2, 3, 4]
print(numbers[:4]) # Output: [0, 1, 2, 3]
print(numbers[5:]) # Output: [5, 6, 7, 8, 9]
print(numbers[:]) # Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(numbers[:2]) # Output: [0, 2, 4, 6, 8]
print(numbers[1:3]) # Output: [1, 4, 7]
print(numbers[-5:-2]) # Output: [5, 6, 7]
```

Common List Methods

Method	Description	Example Code
<code>len(list)</code>	Returns number of elements	<code>len([5, 2, 9]) # 3</code>
<code>append(element)</code>	Adds element to the end of list	<code>nums.append(7)</code>
<code>insert(idx, element)</code>	Inserts element at specific index	<code>nums.insert(1, 4)</code>
<code>sort()</code>	Sorts list in ascending/alphabetical order	<code>nums.sort()</code>
<code>reverse()</code>	Flips the order of elements in list	<code>nums.reverse()</code>

3. Tuples

Tuples are ordered collections of items similar to lists, but they are **immutable**. Because of immutability, operations on tuples are faster.

Indexing, Slicing & Loops

```
# Indexing & Slicing
t = (10, 20, 30, 40)
print(t[0])      # Output: 10
print(t[-1])     # Output: 40
print(t[1:3])    # Output: (20, 30)

# Looping over a Tuple
for val in t:
    print(val)

# Calculating sum of tuple elements
t = (5, 15, 25)
total = 0
for val in t:
    total += val
print("Sum:", total) # Output: Sum: 45
```

Tuple Methods

- `index(val)` - Returns index of first occurrence for `val`.
- `count(val)` - Returns total count of occurrences for `val`.

```
t = (1, 2, 2, 3, 5)
print(t.index(2)) # Output: 1
print(t.count(2)) # Output: 3
```

4. Dictionaries

Dictionaries store data values in key:value pairs. They are ordered, changeable (mutable), and do not allow duplicate keys.

Looping through Dictionary

```
d = {
    "name": "Shradha",
    "subjects": ["math", "science", "physics"],
    "cgpa": 9.5
}

for key, value in d.items():
    print(key, value)
```

5. Sets

A **set** is an unordered collection of unique elements.

Set Characteristics

1. **Unique Elements:** Duplicate values are automatically removed.
2. **Unordered:** No indexing or slicing support.
3. **Mutable:** Elements can be added or removed.

4. Immutable Elements: Items inside a set must be immutable (e.g., strings, numbers, tuples).

```
my_set = {1, 2, 2, 2, 3}
print(my_set)      # Output: {1, 2, 3}
print(len(my_set)) # Output: 3

empty_set = set()  # Empty set creation
```

Set Methods & Mathematical Operations

Method	Description	Example Code
<code>add(val)</code>	Adds an element to the set	<code>s.add(40)</code>
<code>remove(val)</code>	Removes element (raises error if missing)	<code>s.remove(10)</code>
<code>pop()</code>	Removes and returns an arbitrary element	<code>s.pop()</code>
<code>clear()</code>	Removes all elements from set	<code>s.clear()</code>
<code>s1.union(s2)</code>	Returns new set containing all unique values from both sets	<code>A.union(B)</code> # {1, 2, 3, 4, 5}
<code>s1.intersection(s2)</code>	Returns new set containing common values between both sets	<code>A.intersection(B)</code> # {3}

```
s = {10, 20, 30}
s.add(40)      # {10, 20, 30, 40}
s.remove(10)   # {20, 30, 40}

# Union & Intersection
A = {1, 2, 3}
B = {3, 4, 5}

print(A.union(B))      # Output: {1, 2, 3, 4, 5}
print(A.intersection(B)) # Output: {3}
```

✨ Keep Learning & Keep Exploring! ✨

Document created for GitHub profile: [gmlincoln \(Golam Maula Lincoln\)](#)